



MÓDULO 0 · INTRODUÇÃO

Aula 0.1

Preparando o seu Ambiente de Desenvolvimento

Passo a passo, sem pressa, para
Windows e macOS — instale e teste
cada ferramenta do curso.

As 5 ferramentas que vamos montar

Pense que você vai abrir uma marcenaria. Cada tecnologia tem um papel — como cada ferramenta da oficina.



Node.js

O motor do código. É a **energia elétrica** da oficina: sem ela, nenhuma máquina liga.



Git

O **histórico do projeto**. A **máquina do tempo**: tira fotos do seu trabalho e deixa voltar atrás.



VS Code

O **editor de código**. A **bancada de trabalho** onde tudo acontece.



GitHub

Seus projetos na **nuvem**. O **armazém seguro** fora da oficina — e seu portfólio.



GitHub Copilot

A **IA que te ajuda**. Um **colega experiente** sentado ao seu lado o tempo todo.



A ordem certa

1 Node → 2 Git → 3 VS Code → 4 GitHub → 5 Copilot → 🏆 **1ª API**. Faça uma por vez e teste!

Onde digitar os comandos: o Terminal

Vários testes pedem para "digitar no terminal". Ele é a tela preta onde você conversa com o computador por texto.

No Windows

- 1 Aperte a tecla **Windows** (ou clique em Iniciar).
- 2 Digite **PowerShell** (ou "Prompt de Comando").
- 3 Clique no aplicativo que aparecer. Pronto: é ali que você digita.

No macOS

- 1 Aperte **Command (⌘) + barra de espaço** (abre a busca Spotlight).
- 2 Digite **Terminal** e aperte Enter.
- 3 A janela do Terminal abre. É ali que você digita os comandos.



Dica

Para "rodar" um comando, você digita e aperta **Enter**. Se errar, sem problema: é só digitar de novo. Nada quebra por testar.

Node.js – o "motor" do seu código

O que é?

É o programa que permite executar código JavaScript direto no seu computador. Muitas ferramentas do curso só "ligam" se ele estiver instalado.

- **Vem com o npm de brinde:** um "delivery de peças" que baixa pedaços de código prontos para o seu projeto.



Analogia: a energia elétrica

O Node.js é como a **energia elétrica** da casa. Você não a vê, mas é ela que faz a geladeira, a TV e as lâmpadas funcionarem. Sem o Node, é como tentar usar um liquidificador na tomada desligada.



Regra de ouro do download

No site, baixe sempre a versão marcada como **LTS** (estável e recomendada). Evite a versão "Current".

Instalando o Node.js (site oficial: nodejs.org)

No Windows

- 1 Acesse nodejs.org e clique no botão LTS.
- 2 Abra o arquivo baixado (termina em `.msi`).
- 3 Clique em **Next**, aceite os termos e mantenha as opções padrão.
- 4 Clique em **Install** e depois em **Finish**.

No macOS

- 1 Acesse nodejs.org e clique no botão LTS.
- 2 Abra o arquivo baixado (termina em `.pkg`).
- 3 Clique em **Continuar** → **Concordar** → **Instalar**.
- 4 Digite a senha do Mac se for pedida e finalize.

✓ Teste se funcionou (igual nos dois sistemas)

TERMINAL / POWERSHELL

```
$ node --version → v20.11.0  
$ npm --version → 10.2.4
```

Apareceram números de versão? **Deu certo!** 🎉 (Os números podem ser diferentes — o importante é não dar erro.)

Git — a "máquina do tempo" do projeto

O que é?

É um programa que salva o histórico de todas as versões do seu projeto e permite voltar a qualquer momento anterior. Cada "foto" salva se chama **commit**.



Analogia: o fotógrafo do projeto

Imagine tirar uma **foto da página** a cada parágrafo que escreve. Se amanhã bagunçar tudo, é só voltar para a foto de ontem. O Git é esse fotógrafo automático do seu código.



Configuração única (faça uma vez, nos dois sistemas)

```
$ git config --global user.name "Seu Nome Completo"  
$ git config --global user.email "seu.email@exemplo.com"
```

Use o mesmo e-mail que vai usar no GitHub. É como assinar a folha de ponto de cada alteração.

Instalando o Git (site oficial: git-scm.com)

No Windows

- 1 Acesse git-scm.com/downloads e clique em Windows.
- 2 Abra o instalador (.exe).
- 3 Clique em **Next** em todas as telas — as opções padrão são seguras.
- 4 Clique em **Install** e **Finish**.

No macOS

- 1 Abra o **Terminal** e digite `git --version`.
- 2 Se o Git não existir, o Mac pergunta se quer instalar as "ferramentas de linha de comando".
- 3 Clique em **Instalar** e aguarde terminar.
- 4 (Alternativa: baixar pelo site git-scm.com.)

✓ Teste se funcionou (igual nos dois sistemas)

TERMINAL / POWERSHELL

```
$ git --version → git version 2.43.0
```

Apareceu uma versão? **Perfeito!** ✓

VS Code — a sua "bancada de trabalho"

O que é?

É o editor de código: o programa onde você escreve, lê e organiza seus arquivos. Dá para encaixar "acessórios" chamados **extensões** — inclusive o Copilot.



Analogia: a bancada da marcenaria

É onde você apoia o trabalho, tem as ferramentas à mão e boa iluminação. Você ainda personaliza a bancada com acessórios (extensões) com a sua cara.



Não confunda os nomes!

Existe o "Visual Studio" (pago e pesado) e o "Visual Studio Code" (VS Code) — gratuito e leve, o que usaremos. Baixe sempre o VS Code.

Instalando o VS Code (site: code.visualstudio.com)

No Windows

- 1 Acesse code.visualstudio.com e clique no botão azul de download.
- 2 Abra o instalador (.exe).
- 3 Aceite os termos e marque "Adicionar ao PATH" se aparecer.
- 4 Conclua e abra o VS Code.

Teste se funcionou

Abriu a tela de boas-vindas? Crie um arquivo em **Arquivo** → **Novo Arquivo**, escreva algo e salve. Salvou? A bancada está pronta. 

No macOS

- 1 Acesse code.visualstudio.com e baixe (arquivo .zip).
- 2 Abra o .zip — vai surgir o app **Visual Studio Code**.
- 3 Arraste-o para a pasta **Aplicativos**.
- 4 Abra pelo Launchpad. Na 1ª vez, confirme "Abrir".



Deixe em português (opcional)

Aperte **Ctrl/Cmd + Shift + X**, busque "**Portuguese (Brazil) Language Pack**", instale e reinicie.

GitHub – o seu "armazém na nuvem"



Analogia: o cofre na nuvem

Se o Git tira as fotos no seu PC, o GitHub é o álbum guardado em um cofre na internet. Se o PC quebrar, está tudo salvo – e vira seu portfólio.



Dica de carreira

Seu perfil no GitHub é um currículo vivo da área de tecnologia. Cada projeto publicado conta pontos.



Criar a conta (igual nos dois sistemas)

- 1 Acesse github.com/signup.
- 2 Digite o e-mail (o mesmo do Git) e continue.
- 3 Crie uma senha forte e um nome de usuário profissional (ex.: maria-silva-dev).
- 4 Resolva a verificação anti-robô.
- 5 Digite o código enviado ao seu e-mail.
- 6 Escolha o plano gratuito (Free).

✅ Teste se funcionou

Acesse github.com/seu-nome-de-usuario. Abriu o seu perfil? Conta criada! ✅ Guarde bem usuário e senha – vamos usar no próximo passo.

GitHub Copilot – o seu "colega de IA"

O que é?

É uma inteligência artificial que vive dentro do VS Code: sugere código enquanto você digita e conversa com você (o chat) para tirar dúvidas.



Analogia: o colega ao seu lado

Como ter um **colega experiente** sentado ao seu lado. Você explica o que quer, ele sugere e escreve trechos. Mas, como todo colega, pode errar – você revisa e decide.



É de graça para estudantes!

Ative o **GitHub Student Developer Pack** em education.github.com ("Get the Pack"). Se não conseguir, há um **plano gratuito limitado** do Copilot que já dá para começar.

Ativando o Copilot no VS Code



Passo a passo (Windows e macOS)

- 1 No VS Code, abra as **extensões** (Ctrl/Cmd + Shift + X).
- 2 Busque "**GitHub Copilot**" e clique em **Install**.
- 3 Instale também "**GitHub Copilot Chat**".
- 4 Clique em **Sign in** e entre com sua conta do GitHub.
- 5 Autorize no navegador e volte ao VS Code.



Abrir o chat

Clique no balão de conversa na barra lateral ou aperte Ctrl/Cmd + Alt/Option + I.



Teste se funcionou

No chat, escreva: *"Olá! Você está funcionando? Explique em uma frase o que é o GitHub."* Respondeu? 🎉 Para a sugestão de código: crie um **teste.js**, digite **// função que soma dois números** e aperte Enter — aceite a sugestão com **Tab**.

IA, LLM e o chat – com quem você conversa?

Termo	O que é	Analogia
IA (Inteligência Artificial)	A área que faz máquinas realizarem tarefas que pareciam exigir inteligência humana.	O "guarda-chuva" geral, como dizer "transporte".
LLM (Modelo de Linguagem)	IA treinada lendo uma enormidade de textos; aprendeu a prever e escrever palavras. O Copilot usa um LLM por trás.	Um transporte específico, como "o carro".
Chat / Prompt	O prompt é a instrução que você escreve; o chat é a conversa de ida e volta.	Você dando as direções ao motorista.



Como a IA "pensa", em uma frase

Um LLM é um **autocompletar superpoderoso**: prevê a próxima palavra com base em padrões. Por isso é ótimo para escrever e explicar – mas não **"sabe"** a verdade e às vezes inventa com confiança (a "alucinação"). Sempre confira o que for importante.

O que pedir ao chat — e como pedir bem

O que dá para pedir

-  **Escrever código:** "Crie uma função que valide um e-mail."
-  **Explicar:** selecione um trecho e pergunte "o que isto faz?"
-  **Achar erros:** "Por que dá erro? Como corrijo?"
-  **Criar testes:** "Escreva testes para esta função." (puro QA!)
-  **Aprender:** "Explique como se eu fosse iniciante."

A regra de ouro: seja específico

 Fraco	 Bom
"Faça um código."	"Crie em JavaScript uma função que receba uma lista de números e devolva a média, com comentários."
"Está dando erro."	"Dá o erro X. Aqui o código: [cole]. O que está errado e como corrijo?"



Receita C.O.F.

Contexto + Objetivo + Formato. Ex.: "Estou aprendendo testes (contexto). Crie 3 casos de teste de login (objetivo). Em formato de tabela (formato)."



Cuidados

Nunca cole senhas ou dados de clientes no chat. Sempre revise e teste o que a IA gerar. Se não ficou bom, refaça com mais detalhes (isso é "iterar").

Crie a sua primeira API com o Copilot

O objetivo é dar o primeiro passo do seu primeiro aplicativo. Mas, antes, o que é uma API? Pense em uma casa. 🏠

Na casa 🏠	Na API 💻	O que faz
O endereço da casa	Rota principal (URL base) – <i>meuapp.com</i>	É onde a casa "fica". Quem quiser falar com o app precisa saber o endereço.
As portas e janelas	Endpoints (entradas) – <i>/usuarios, /login</i>	Cada porta leva a um cômodo. Cada endpoint atende a um tipo de pedido.
A chave da porta	Token (autenticação)	Só quem tem a chave entra. O token prova que você tem permissão.
O que se pede na porta	Requisição e resposta	Você bate na porta pedindo algo e recebe a resposta de volta.



Resumo da analogia

Para visitar alguém você precisa do **endereço** (rota), entrar pela **porta certa** (endpoint) e ter a **chave** (token). Uma API funciona igualzinho: outros programas "visitam" o seu app seguindo essas regras.

Sua missão e o prompt de exemplo

Sua missão

- 1 Escolha o app que quer criar (sugestões no próximo slide).
- 2 Abra o chat do Copilot.
- 3 Use o prompt ao lado, adaptando o que está em [colchetes].
- 4 Peça ao Copilot para **explicar** o que não entender.
- 5 Rode a API seguindo as instruções que ele der.

PROMPT DE EXEMPLO — COPIE, COLE E ADAPTE

Estou começando a programar e quero criar a minha primeira API simples.

Contexto: estou aprendendo testes de software e quero um app de [DESCREVA SEU APP, ex.: "lista de tarefas do curso"].

Por favor:

1. Crie uma API básica em Node.js com Express, com endpoints para: listar itens, adicionar e remover.
2. Use dados em memória (sem banco de dados por ora).
3. Explique, com comentários simples, o que é cada endpoint (a "porta"), a rota principal (o "endereço") e onde eu colocaria um token (a "chave") no futuro.
4. Me diga o passo a passo para rodar e testar.

Capriche nas explicações, é a minha primeira vez. 😊



Depois que funcionar, evolua!

Peça: *"Agora escreva alguns testes automatizados para essa API"* — assim você já pratica QA, o foco do nosso curso.

Ideias de aplicativos para o seu projeto

Escolha algo que te empolgue! Apps ligados a 2026 ou que ajudam nos seus estudos.

Inspirados em 2026

-  **Companheiro da Copa 2026:** tabela de jogos, palpites e contagem regressiva.
-  **Guia das Eleições 2026:** calendário, lembretes e checklist de documentos.
-  **Roteiro de Verão / Floripa:** praias e pontos turísticos com avaliações.
-  **Controle de Gastos 2026:** registre receitas e despesas do ano.

Para ajudar nos estudos

-  **Lista de Tarefas do Curso:** a mais simples — perfeita para começar.
-  **Registrador de Bugs:** descrição, gravidade e status. Puro QA!
-  **Flashcards do CTFL:** cartões de pergunta e resposta para revisar.
-  **Cronômetro Pomodoro:** ciclos de estudo e pausa, contando suas horas.



Sem ideia ainda?

Comece pela "Lista de Tarefas do Curso": é a mais simples e ensina tudo que você precisa. Depois é só usar o mesmo aprendizado para algo mais ambicioso.

DEU ALGUM PROBLEMA?

Respire e tente isto

- **Comando "não reconhecido"?** Feche e abra o terminal de novo (às vezes ele só precisa reiniciar para enxergar o programa novo). Se persistir, reinicie o computador.
- **O Copilot não responde?** Verifique se você está logado com a conta do GitHub e se a extensão está instalada e ativa.
- **Travou numa etapa?** Copie a mensagem de erro e cole no chat do Copilot: "O que significa este erro e como resolvo?"
- **Ainda travado?** Anote em que passo parou e leve ao grupo do curso. Errar faz parte — todo programador já passou por isso! ❤️



Parabéns por chegar até aqui!

Você acabou de montar a sua oficina:

Node.js

Git

VS Code

GitHub

Copilot

Sua 1ª API

Agora é só criar! 🚀